

Full length article

Losing control: Exposing security weaknesses of Kubernetes control plane interfaces

Chen Wang^{ID}, Hongbo Tang^{ID*}, Yu Zhao, Wei You, Jie Yang, Hang Qiu

Information Engineering University, Zhengzhou 450002, Henan, China

ARTICLE INFO

Keywords:

Kubernetes
Control plane interfaces
Control plane interface attacks
DoS

ABSTRACT

Kubernetes has become the dominant container orchestration platform, relying on a centralized control plane to manage workloads across nodes. However, its exposed control plane interfaces introduce critical security risks. This paper conducts a systematic static analysis of these interfaces and uncovers insufficient access controls and missing rate-limiting mechanisms. We design four attack strategies and implement seven representative attacks in both local and cloud environments. The experiments demonstrate severe consequences, including sensitive data leakage, denial-of-service conditions, up to 90% CPU overhead, and 70% packet loss in co-located containers, which also lose the ability to resolve non-local DNS queries. Based on these findings, we propose mitigation strategies that have been acknowledged by cloud vendors and the Kubernetes community, with plans for deployment in future releases. This work provides the first systematic study of control plane interface vulnerabilities, and future research should explore automated analysis frameworks and isolated experimental environments to strengthen Kubernetes security in multi-tenant commercial platforms.

1. Introduction

Kubernetes is a container orchestration system for automating deployment, scaling, and management of containerized applications. In recent years, Kubernetes has been widely adopted by multiple companies to manage their workloads and is used by cloud vendors as container orchestrators, such as Amazon, Google, Microsoft and Alibaba. According to the CNCF (Cloud Native Computing Foundation) survey, 93% of companies are either using Kubernetes in production, piloting it, or actively evaluating it (CNCF, 2025a). Therefore, Kubernetes has become the de facto standard for container orchestration.

With its wide deployment, Kubernetes also raises a number of security and privacy concerns (Yang et al., 2021b), especially the co-residency attacks in Kubernetes cluster (Shringarputale et al., 2020), DDoS attacks against Kubernetes' auto-scaling (David, 2021), misconfigurations of Kubernetes security practices (Islam Shamim et al., 2020; Shamim, 2021; Haque et al., 2022), privilege escalation in Kubernetes (Pecka et al., 2022), and excessive permissions granted to third-party applications (Yang et al., 2023).

Note that a Kubernetes cluster consists of a control plane and one or more worker nodes. The control plane is the centralized control of the cluster, where the necessary Kubernetes components and a number of third-party applications (or apps) are running. All the worker nodes are managed by the control plane and they contain the components required to run the containers, i.e., kubelet, kube-proxy, and the

container runtime. Meanwhile, on the control plane, kube-apiserver (a core component of Kubernetes) and third-party applications expose various interfaces for providing services. We call these interfaces *control plane interfaces*. The control plane interfaces are instrumental to the whole cluster working properly. However, the security of the control plane interfaces has not been systematically studied before. Prior studies have identified critical security weaknesses in cloud-native control planes, including insufficient authorization mechanisms, inadequate access controls, and the absence of fine-grained isolation policies—particularly in multi-tenant environments (Addula and Ali, 2025; Almanasir et al., 2025). This paper employs static analysis to evaluate the security mechanisms of control plane interfaces in Kubernetes, revealing insufficient access restrictions and lack of rate limiting. Consequently, a malicious container may issue non-restricted or rate-unlimited requests – either directly or indirectly – to these interfaces, potentially triggering what we term *control plane interface attacks*. Drawing on risk analyses based on the NIST (National Institute of Standards and Technology) Cybersecurity Framework and recent advances in secure cloud access control (Mohd Satar et al., 2025; Aljumaiah et al., 2025), we argue that such attacks can lead to severe consequences, including disruption of control plane functionality, excessive resource consumption, leakage of sensitive cluster information, and cross-tenant impact on co-located victim workloads. Our focus is on scenarios where a Kubernetes cluster is shared by multiple tenants, with an attacker's

* Corresponding author.

E-mail address: tahobo@sina.com (H. Tang).

pod and victim pods running under different principals but sharing the same control plane.

To systematically identify potential exploits for control plane interfaces, we identify four strategies (direct vs. indirect) targeting various access paths to these interfaces. To further reveal the security risks associated with these attack strategies, we conduct seven attacks across four case studies. These attacks are performed in a local environment as well as in two cloud-vendor environments. In our local environment, the attacks disrupt the functionality of the control plane and other containers within the cluster. Specifically, the control plane leaks the cluster's topology and several resource quotas, which could potentially be exploited for further attacks. Additionally, the control plane randomly drops packets and consumes up to 90% of the CPU resources on the control plane node, as measured using the `top` command. Furthermore, other containers experienced noticeable difficulties in resolving non-local DNS queries and encountered up to 70% packet loss when performing ICMP ping tests to the kube-apiserver.

We conducted experiments in both Alibaba Cloud and Taikun Cloud, similar to those performed in our local testbed. Our findings reveal that both cloud providers are susceptible to control plane interface attacks. We ensured that our experiments would not have a real-world impact by requesting the cloud vendors to provide a dedicated cluster for our testing, which allowed us to monitor the control plane's status without disrupting actual services. According to the CNCF cloud vendor list (CNCF, 2025b), Alibaba Cloud and Taikun Cloud are the only providers that meet our ethical requirements for such testing. We find four attacks succeed on Alibaba Cloud and five on Taikun Cloud, reflecting variations in default settings, security measures, and component choices. We believe that other cloud vendors are also vulnerable to these control plane interface attacks, particularly if they operate multi-tenant workloads within a single Kubernetes cluster.

We report these problems to the related communities and cloud vendors. For communities, we report these issues to the Kubernetes community and third-party app communities. They confirm all these problems, and they fix the problems immediately or plan to fix them. For cloud vendors, we report these issues to Alibaba Cloud and Taikun Cloud. Alibaba Cloud confirms all the bugs and provides multiple suggestions. And Taikun also confirms all the problems. As well, they will add additional restrictions and make upgrading to their productions to fix these problems. At last, we propose several strategies for mitigating the risks of control plane interface attacks.

In general, the major contributions of this work are summarized as follows.

New Attack Surface. We reveal that quite a number of control plane interfaces of Kubernetes have no proper access restrictions or rate limitations. Thus, a malicious container can exploit *control plane interface attacks*, resulting in information leaks, control plane disruptions, compromised functionality of other containers, and increased resource consumption by the control plane.

Attacking Strategies. We systematically explore the strategies to attack the control plane interfaces. For direct access, a malicious container can send unrestricted or rate-unlimited requests to the interfaces of kube-apiserver and third-party apps. For indirect access, a malicious container can leverage the kubelet as a proxy to send certain (malicious) requests to the kube-apiserver, or leverage the kube-apiserver as a proxy to send certain requests to all other nodes in the whole cluster.

Practicality Evaluation. We conduct seven attacks in four case studies to evaluate the impacts. Due to these attacks, the control plane leaks the cluster's sensitive information, drops network packets randomly, and consumes up to 90% of CPU resources. At the same time, the victim container suffers up to 70% network packet loss, and cannot send non-local DNS queries. The local environment, Alibaba Cloud, and Taikun Cloud are all vulnerable to our attacks. We report these problems to the

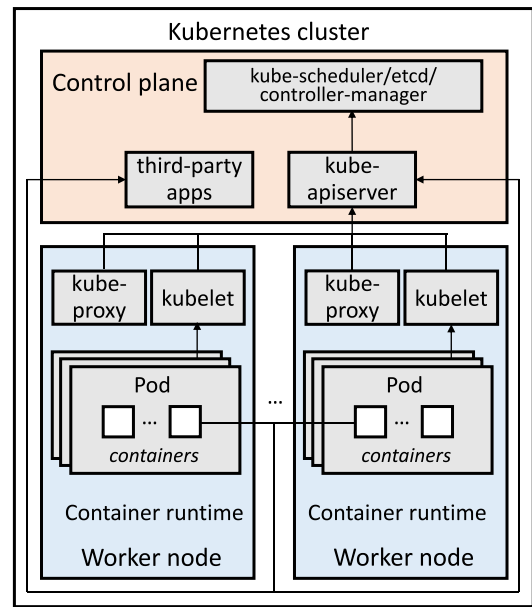


Fig. 1. The overview of Kubernetes architecture.

related communities and cloud vendors. They all confirm the issues and plan to fix them.

The rest of this paper is structured as follows. We introduce the necessary background knowledge of Kubernetes in Section 2. We present a real example as motivation and give out four strategies for attacking the control plane interfaces in Section 3. We use four case studies in both the local and the cloud-vendor environments to show that the attack surface is practical in Section 4. We discuss the practicality of these attacks and measures to mitigate these issues in Section 5. We summarize the related work in Section 6 and conclude our paper in Section 7.

2. Background

In this section, we present the necessary background knowledge of Kubernetes, including the Kubernetes architecture and its application scenarios.

Kubernetes architecture. Kubernetes is a container orchestration system that provides a framework for creating and managing containers at scale. As shown in Fig. 1, a Kubernetes cluster consists of a control plane and one or more worker nodes. Particularly, the control plane is responsible for managing and maintaining the clusters. There are usually four Kubernetes components (i.e., kube-apiserver, kube-scheduler, etcd and controller-manager) and multiple third-party applications (or apps) on the control plane. The worker node, which can be a physical or virtual machine, is responsible for running containers. On the worker node, there are three components (i.e., kubelet, kube-proxy, and container runtime) and multiple pods, which are groups of one or more containers.

Further, on the control plane, besides the Kubernetes components, a number of third-party apps (a.k.a, addons (Kubernetes Authors, 2025c)) extend the functionalities of Kubernetes. Accordingly, there are four main types of third-party apps, i.e., networking and network policy, service discovery, visualization & control, and infrastructure. For example, Calico is a networking and network policy provider, and one of its components, calico-kube-controller runs on the control plane to implement the network policies. For service discovery, CoreDNS is a flexible and extensible DNS server that implements the Kubernetes DNS specification by default. For visualization & control, Grafana is the most popular Kubernetes dashboard UI (user interface). While for

infrastructure, Prometheus is an open-source monitoring system for which Grafana provides out-of-the-box support. As one of Prometheus' components, the `node_exporter` runs on each node of the Kubernetes cluster to collect the status information of all the nodes.

Application scenarios. There are a large number of Kubernetes platforms deployed in the business environments, which are divided into four types according to [CNCF \(2025c\)](#), i.e., distribution, installer, hosted, and PaaS(Platform as a Service)/container service. Specifically, the Kubernetes distributions are specified Kubernetes packages provided by different vendors, and the Kubernetes installers are tools to help install Kubernetes by one step on machines. These two former types of Kubernetes platforms are used by many companies to deploy and manage their own Kubernetes clusters, such as AppDirect, MAK IP, Travel-Wallet, Alibaba Travels, and so on. Further, the hosted Kubernetes are multiple Kubernetes services provided by cloud vendors, which allow tenants to spin up a Kubernetes cluster on demand. Kubernetes clusters are managed by the cluster vendors in their infrastructures. For example, Alibaba Cloud ACK (Alibaba Cloud Container Service for Kubernetes) deploys and manages Kubernetes clusters on Alibaba Cloud ECS (Alibaba Cloud Elastic Compute Service) virtual machines. Taikun is another Kubernetes vendor that deploys and manages Kubernetes on infrastructures specified by tenants. Moreover, PaaS/container services allow users to run applications without considering the details of the underlying compute resources and infrastructures, which are deployed in a number of cloud vendors, such as Akka, Heroku, clever cloud, and so on.

Ideally, the containers and worker node components only interact with the control plane of Kubernetes necessarily to keep the cluster running as expected. However, the control plane exposes multiple service interfaces to the containers and Kubernetes components without proper access restrictions or rate limitations. Our key observation in this paper is that a malicious container is capable of sending a large number of (access-unrestricted or rate-unlimited) requests to these interfaces directly or indirectly, which lead to functionality disruption and extra resource consumption of the control plane, sensitive information leakage (e.g., the cluster's topology), and even other containers' functionality disruption.

3. Motivation and attacking strategies

In this section, first we describe the threat model and assumptions of our work. Next, we present the motivation of this paper and use a real example to illustrate the details. Finally, we introduce four attacking strategies to exploit the new attack surface.

3.1. Threat model and assumptions

In this paper, we assume that an attacker can create and control several containers (a.k.a, malicious containers) in the Kubernetes cluster, and attempts to disrupt the control plane and other users' containers (a.k.a, victim containers) in the same cluster. The attacker's containers can access control plane service interfaces over the cluster network, as is default in most deployments ([Kubernetes Authors, 2025a,i](#)). As we are targeting the Kubernetes control plane, different users' containers are running in the same Kubernetes cluster and sharing one control plane.

We also assume that the malicious containers are restricted by the most secure practices. Specifically, we follow the Kubernetes security recommendations ([Kubernetes Authors, 2025g](#)) to harden the cluster and restrict the containers. For hardening the cluster, we control accesses to the Kubernetes APIs by using TLS (Transport Layer Security), API authentication & authorization, etc. In addition, we control access to the kubelet by denying unauthorized requests. Further, we control the capabilities of a workload or user at runtime and protect the cluster components by restricting accesses to the `etcd`, rotating infrastructure credentials frequently, etc.

For restricting the containers, we assume that the malicious containers are isolated by Kubernetes namespaces and running on one separate worker node. In particular, on a worker node, the containers run as non-root users with all capabilities dropped. At the same time, the worker node's OS kernel enforces as many Linux namespaces and control groups as possible for the container. Moreover, the worker node kernel leverages `seccomp` to block sensitive system calls. We further assume that all the worker nodes' kernels have no bugs and all the security mechanisms are working correctly. An attacker is able to run any code in the malicious containers and invoke `seccomp`-allowed system calls. However, the attacker is not allowed to explore kernel vulnerabilities.

In the following, we use a real example to show that due to the access-unrestricted and rate-unlimited control plane interfaces, even with all the above security and protection enforced, malicious containers can still initiate practical attacks on the Kubernetes control plane, resulting in the leakage of sensitive information, disruptions to the functionalities of other containers and the control plane itself, and increased resource consumption.

3.2. Motivation example

In this section, we use the CoreDNS app as an example to show that a malicious container can send non-restricted and rate-unlimited requests to the control plane interfaces exposed by the CoreDNS service, which has two consequences. First, a malicious container gets the cluster's sensitive information to disrupt other containers' non-local DNS queries. Second, a malicious container can fill up the control plane's `nf_conntrack` table, disrupting the control plane's functionalities and consuming more resources.

CoreDNS implements the Kubernetes DNS specification, and it is practically the default DNS service application in the Kubernetes cluster. It is widely deployed in Kubernetes clusters both in-house and by cloud vendors. Specifically, CoreDNS exposes the 9153 TCP port on the control plane to provide service metrics information. All the containers in the Kubernetes cluster are able to access this control plane interface without any access restrictions or rate limitations. Thus, when a malicious container sends certain (malicious) requests to CoreDNS-SERVICE-IP: 9153/metrics, it possibly leads to unexpected consequences.

First, CoreDNS leaks sensitive information, which can be leveraged by a malicious container to attack the CoreDNS service and disrupt other containers' functionalities. More specifically, the CoreDNS metrics return `MaxConcurrentRejectCount`, which is a counter of rejected concurrent non-local DNS queries. If the number of concurrent queries exceeds the handling capacity of CoreDNS, the `MaxConcurrentRejectCount` will be increased to a non-zero value. As shown in [Fig. 2](#), when CoreDNS receives non-local DNS queries, the `ServeDNS()` function increases `f.concurrent` (line 4), and compares `f.concurrent` with `f.maxConcurrent` (line 6). If the number of current queries is out of this quota, it returns an error and increases `MaxConcurrentRejectCount` (line 7). Thus, a malicious container can send a large number of concurrent queries to coreDNS. At the same time, it leverages `MaxConcurrentRejectCount` to observe whether `f.concurrent` reaches `f.maxConcurrent`. Once the counter becomes a non-zero value, it denotes that the concurrent non-local DNS queries reach `f.maxConcurrent`, and CoreDNS will deny the following non-local DNS queries of other containers in the cluster.

Second, a malicious container can send a large number of rate-unlimited requests to the CoreDNS metrics service, which fill up the control plane's `nf_conntrack` table and disrupt the control plane's functionalities. Specifically, the Linux kernel's networking stack uses connection tracking to keep track of all logical network connections or flows, and the kernel maintains a table (i.e., the `nf_conntrack` table) to record the detailed information of each connection. The total number of connections has a limit, i.e., `nf_conntrack_max`. If one generates a

```

1      func (f *Forward) ServeDNS(ctx context.Context,
2      ↪ w dns.ResponseWriter, r *dns.Msg) (int,
3      ↪ error) {
4          ...
5          if f.maxConcurrent > 0 {
6              count := atomic.AddInt64(&(f.concurrent), 1)
7              defer atomic.AddInt64(&(f.concurrent), -1)
8              if count > f.maxConcurrent {
9                  MaxConcurrentRejectCount.Add(1)
10                 return dns.RcodeRefused,
11                     ↪ f.ErrLimitExceeded
12             }
13         }
14     }
15 }

```

Fig. 2. The source code of ServeDNS. The `f.concurrent` (line 4) is the counter of concurrent queries per second, and the `f.maxConcurrent` is the quota of concurrent queries per second. It compares the `f.concurrent` with `f.maxConcurrent` (line 6). If the check fails, it adds the `MaxConcurrentRejectCount` (line 7) and returns an error.

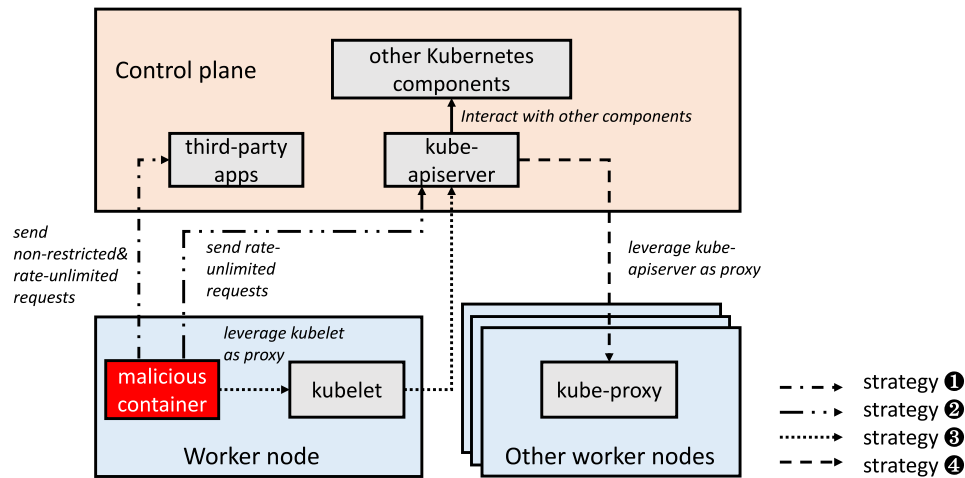


Fig. 3. The overview of four exploiting strategies to attack cluster from a malicious container.

large number of TCP connections on the control plane in a short time, it will consume all the quota of the `nf_conntrack` table, which causes the control plane's Netfilter module malfunction. Thus, a malicious container can send rate-unlimited requests to the CoreDNS-SERVICE-IP:9153/metrics to cause a DoS attack. Our experiments in a local testbed show that the attack makes the control plane drop network packet randomly, which disrupts its functionality. At the same time, it consumes additional CPU resources (see details in Section 4.2).

We report this issue to the CoreDNS community, Alibaba Cloud and Taikun Cloud. They all confirm this problem. Further, for the CoreDNS community, we suggest fixing this problem by securing the metrics interface using HTTPS. They accept our recommendation. For Alibaba Cloud, as their best practices follow the community, the problem will be naturally fixed with the CoreDNS community. Taikun Cloud plans to fix this problem by adding more restrictions, and it will be ported to their production environments.

3.3. Attacking strategies overview

To systematically find ways to attack the control plane interfaces, we design four strategies according to different access paths of control plane interfaces, as shown in Fig. 3. For each strategy, we present an approach to perform the attacks. First, a malicious container can send non-restricted and rate-unlimited requests to third-party app interfaces directly to perform attacking, i.e., strategy ① in Fig. 3. Second, a malicious container can send rate-unlimited requests to kube-apiserver interfaces directly, and make it interact with other Kubernetes components for attacking, i.e., strategy ② in Fig. 3. Third, a malicious

container can leverage kubelet as a proxy to send certain requests to the kube-apiserver's service interfaces which are not allowed to access directly, and make the kube-apiserver interact with other Kubernetes components to achieve attacks, i.e., strategy ③ in Fig. 3. Fourth, a malicious container can leverage the kube-apiserver as a proxy to send a large number of requests to the kube-proxy on other worker nodes for jumping attacks, i.e., strategy ④ in Fig. 3.

3.4. Sending requests to third-party apps directly

As shown in Fig. 3, the first attacking strategy is to send non-restricted and rate-unlimited requests to the control plane interfaces exposed by third-party apps. Multiple third-party apps on the Kubernetes control plane expose a number of service interfaces without access restrictions or rate limitations. Thus, a malicious container can send a large number of requests to these interfaces directly, which might lead to information leakage and DoS attacks.

For example, as described in Section 3.2, CoreDNS exposes a metrics interface that can be accessed by all containers in the whole cluster. A malicious container can send certain non-restricted or rate-unlimited requests to this interface for an attack. Besides CoreDNS, several other third-party apps, e.g., node exporter, Grafana, and calico-kube-controller on the control plane, also expose service interfaces without access restriction or rate limitation. Thus, these interfaces can all be exploited by a malicious container with the same strategy. Finally, a malicious container can get sensitive information, disrupt the control plane's functionalities, and consume more resources. We present more details in Section 4.2.

Table 1

The control plane interfaces and corresponding verbs exposed by kube-apiserver we use in Sections 3.6 and 3.7.

Components	Interfaces	Verbs	Impacts
Kubelet	Pods/status	patch	Send pod's status changes
Kube-proxy	services	watch	Watch service changes
	endpoints	watch	Watch endpoints changes

3.5. Sending requests to kube-apiserver directly

As shown in Fig. 3, the second attacking strategy is to send a massive number of rate-unlimited requests directly to the kube-apiserver interfaces (Kubernetes Authors, 2025h). The kube-apiserver exposes multiple interfaces, and it uses the role-based access control (RBAC) policy to restrain which interfaces the containers and other components can access. However, the RBAC policy only restricts the interfaces that can be accessed by the container in a single request, but it does not limit the frequency of access to the interfaces. Thus, a malicious container can send a large number of requests to those interfaces which permit access in a short time. After that, to process these requests, the kube-apiserver interacts with other Kubernetes components, which can be exploited to make DoS attacks.

For example, each container has a default service account token, which is an identity for processes that run in a container. Based on the token, RBAC permits the container to access particular interfaces of the kube-apiserver. A malicious container can leverage the default user token to send a large number of requests to multiple kube-apiserver interfaces. After that, the requests to certain interfaces provoke the kube-apiserver to send messages to other components to accomplish the processing. As a result, the kube-apiserver and other Kubernetes components might fill up the control plane's nf_conntrack table and consume more CPU resources considerably. Moreover, the victim containers possibly drop network packets significantly when connecting to the control plane. We discuss more details in Section 4.3.

3.6. Leveraging Kubelet as proxy

As shown in Fig. 3, the third attacking strategy is to leverage the kubelet as a proxy to send malicious requests to kube-apiserver interfaces indirectly, which consists of two steps. First, a malicious container uses the kubelet as a proxy to send a large number of requests to interfaces that kube-apiserver does not allow containers to access. After that, to handle the incoming requests, the kube-apiserver interacts with other components and consumes resources considerably.

To achieve the first step, one feasible way is via the container's pod status changes, which provides a simple method to make the container invoke kubelet to send requests. More specifically, the kubelet synchronizes the pod's status changes to the kube-apiserver, and has patch verbs on pods/status interfaces (as shown in Table 1), which containers' service account tokens usually cannot access. Thus, a malicious container can change its pod status and leverage the kubelet as a proxy to send patch verbs to the pods/status interfaces indirectly for the attack.

To achieve the second step, one available approach is to exploit the complicated dependencies of the incoming data. More specifically, once the kube-apiserver receives pod status changes, it modifies the status of all the workload (Kubernetes Authors, 2025j) resources related to the pod, not just the pod status changes themselves. Thus, the kube-apiserver sends requests to the etcd component to update all these data structures. In other words, the data structure dependencies of the incoming request amplify the effect of the attack, which causes the kube-apiserver and etcd to consume additional control plane resources vastly. We describe more details in Section 4.4.

3.7. Leveraging kube-apiserver as proxy

As shown in Fig. 3, for the fourth attacking strategy, a malicious container can exploit the kube-apiserver to send a huge number of messages to the kube-proxy on all other nodes for attacking. This strategy can be split into two stages. First, the malicious container sends requests to the kube-apiserver in some way. After that, the treatment logic of this request asks the kube-apiserver to send messages to the kube-proxy on other worker nodes.

We have presented the approaches to achieve the first stage in Sections 3.5 and 3.6. To achieve the second stage, one feasible solution is to leverage the watch mechanism of kube-proxy. More specifically, the kube-proxy has watch verbs on services and endpoints interfaces (as shown in Table 1). It watches the services/endpoints status changes sent by the kube-apiserver, and then modifies the items of the worker node iptables according to the received status change messages. A malicious container can use the third attacking strategy (④ in Fig. 3) to send the pod's status changes to the kube-apiserver. After that, to handle the incoming requests, the kube-apiserver sends related status change messages (e.g., the service/endpoint status changes related to the pod) to the kube-proxy of all the other worker nodes, and the kube-proxy flushes each node's iptables. In other words, a malicious container can leverage the kube-apiserver as a proxy to send messages to all the other worker nodes in the Kubernetes cluster, which will flush all the node's iptables. We give out more details in Section 4.5.

4. Attacks on Kubernetes

In this section, we conduct seven attacks in four case studies to show that the potential attacking strategies are practical. We summarize each attack in Table 3. In each attack, we present the attack steps and the details of the experiments.

4.1. Ethical considerations and environment setup

Ethical considerations. We do the experiments in both the local and the cloud-vendor environments. For the cloud-vendor environments, we intend to evaluate the attacks without causing real-world impact. As attacking control plane interfaces will inevitably generate cluster-level implications, which potentially affect the control plane and all the containers' functionalities in the same cluster, so it is unethical to deploy containers in multi-tenant cloud-vendor environments.

More specifically, the cloud environments should guarantee two points. First, we are the only tenant in the cluster. Second, as we are targeting the control plane of Kubernetes, the Kubernetes cluster should allow us to observe the status of Kubernetes's control plane and all other worker nodes. As a result, we select Alibaba Cloud ACK and Taikun Cloud from the CNCF's hosted Kubernetes vendor list. To the best of our knowledge, they are the only two cloud vendors that fulfill both of our requirements.

Local environment. In our local testbed, we deploy a three-node cluster using kubeadm (Kubernetes Authors, 2025d). The cluster's version, topology, node's configuration, and third-party apps are summarized in Table 2.

We follow the Kubernetes security practices (Kubernetes Authors, 2025g) to harden the Kubernetes cluster, and follow the Kubernetes pod hardening best practices (Kubernetes Authors, 2025e) to restrict containers by using multiple pod-level configurations. To harden the cluster, we control access to the kube-apiserver by using TLS, API authentication & authorization, and disabling anonymous access. In addition, we restrict access to the etcd, rotating infrastructure credentials frequently, and denying unauthorized access to the kubelet.

To restrict containers, we run different users' containers in separate Kubernetes namespaces by setting namespace. We constrain

Table 2

Clusters used for evaluation.

Cluster	Distribution	Topology	VM	OS	Containerd	Third-party Apps
Local	1.30.4	1 control plane, 2 worker	4 core, 8 GB	Ubuntu 22.04	1.7.21	CoreDNS, Prometheus, Grafana, Calico
Alibaba Cloud	1.22.3-aliyun.1	3 control plane, 2 worker	4 core, 8 GB	Ubuntu 20.04	1.5.10	CoreDNS, Prometheus, Flannel
Taikun Cloud	1.21.5	1 control plane, 2 worker	4 core, 8 GB	Ubuntu 20.04	1.5.10	CoreDNS, Prometheus, Flannel

Table 3

Summary of the case studies in different environments. “Y” indicates a successful attack, “N/A” indicates this attack is not applicable to the environment, and “N” indicates a failed attack.

Case	Strategy	Local	Alibaba	Taikun	Attacking impacts
Third-party apps	CoreDNS	Y	Y	Y	Disrupting DNS queries; dropping 40% packets; consuming 90% CPU
	Node Exporter	Y	N	Y	Leaking control plane status; dropping 40% packets; consuming 80% CPU
	Grafana	Y	N/A	N/A	Victim drops 70% packets; consuming control plane 79% CPU
	Calico	Y	N/A	N/A	Leaking cluster's topology; dropping 50% packets; consuming 80% CPU
Kube-apiserver attacks	②	Y	Y	Y	Victim drops 50% packets; components consume control plane 90% CPU
Readiness probe attack	③	Y	Y	Y	Consuming control plane 55.3% CPU
Endpoints flooding attack	④	Y	Y	Y	Consuming control plane 60% CPU; flushing all nodes' iptables

different users' containers running on separate worker nodes by setting nodeName. On each worker node, we run containers with non-root user by setting runAsUser and runAsGroup to 1000. We disable privilege escalation by setting allowPrivilegeEscalation:false and privileged:false. We drop all capabilities. Further, the worker node's kernel enforces as many Linux namespaces and control groups as possible for the container, and it uses RuntimeDefault seccomp profile (Kubernetes Authors, 2025f) to block sensitive system calls. Besides, we use more restrictions such as disallowing host namespaces, host ports, host path volumes, etc.

Alibaba Cloud. In the Alibaba Cloud environment, we use the Alibaba Cloud ACK to deploy a Kubernetes cluster. We also summarize our installation configurations in Table 2. For securing the Kubernetes cluster, as ACK installs and hardens the cluster following its own configurations automatically at the cluster level, we only add more restrictions to secure the containers. More specifically, the ACK user guides of multiple container service deployments only restrain containers with Privileged Container: false and ResourceLimits.

Taikun Cloud. Taikun Cloud is a Kubernetes cloud vendor which deploys and manages the Kubernetes cluster on a virtual machine provider specified by the tenant. In the Taikun Cloud, we choose three Azure VMs to deploy the Kubernetes service, and we also summarize the installation configurations in Table 2. For securing the Kubernetes cluster, similar to the Alibaba Cloud, Taikun deploys and hardens the cluster following its own configurations at cluster level, and we add more restrictions to secure the containers. More specifically, the Taikun Cloud does not provide any official guide for container restrictions. Thus, we adopt a more robust security policy, which is the same as our local environment.

4.2. Case 1: Third-party application attacks

The first case is the third-party application attacks. A malicious container can send non-restricted and rate-unlimited requests to these third-party app interfaces on the control plane directly (① in Fig. 3), which lead to several consequences.

First, these third-party apps leak the count and quota of control plane's nf_conntrack table, cluster's topology, pods number of each node, and the reject count of non-local DNS queries. Some of this information can be leveraged to attack the third-party apps themselves and disrupt other containers' functionalities.

Second, a malicious container can send a large number of requests to these third-party applications, causing them to fill up the control plane's nf_conntrack table and consume up to 90% more CPU resources.

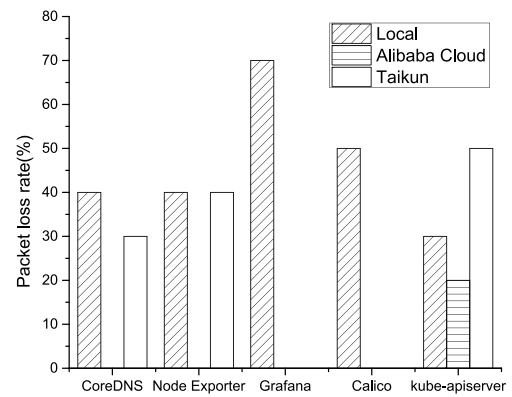


Fig. 4. The packet loss rate of kube-apiserver and four third-party apps attacks. Grafana and Calico are not applicable to Alibaba Cloud and Taikun Cloud, and node exporter is failed on Alibaba Cloud, thus the corresponding columns are empty.

In addition, the victim container suffers up to 70% network packet loss when connecting to the kube-apiserver.

Attacking CoreDNS. We have discussed the steps and consequences for attacking CoreDNS in Section 3.2. In this section, we show the experiment details in different environments. Specifically, in our local environment, a malicious container can send many non-local DNS queries to the service interface of CoreDNS, and it only takes several seconds to make `f.concurrent` (as shown in Fig. 2) reaches the quota. Thus, the `MaxConcurrentRejectCount` returned by metrics becomes a non-zero number, and other containers cannot send non-local DNS queries. Further, we use seven malicious containers on one worker node to send a large number of requests to the CoreDNS pods, which fill up the control plane's `nf_conntrack` table, and the control plane drops network packets randomly. In addition, the victim container suffers 40% packet loss when connecting to the control plane, and the CoreDNS pods consume 90% control plane CPU resources.

On the Alibaba Cloud ACK, CoreDNS pods do not run on the control plane by default, and the `nf_conntrack` table attack fails on Alibaba Cloud. However, a malicious container can still interact with the CoreDNS metrics service to get sensitive information, and then it leverages the leakage information to attack the CoreDNS service itself. Specifically, a malicious container can send a large number of non-local DNS queries, which reach the quota of concurrent queries forwarded to the upstream resolver. Thus, other victim containers' non-local DNS queries are disrupted by this attack, which results in a DoS attack.

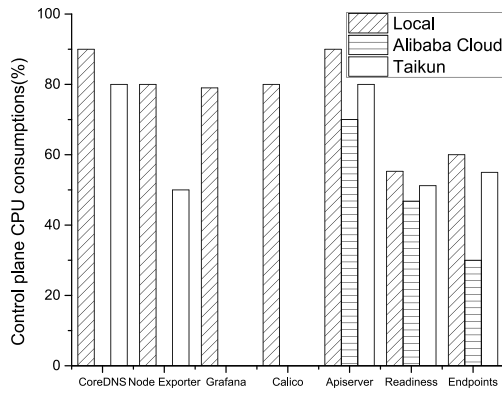


Fig. 5. The control plane CPU consumptions of all seven attacks.

On the Taikun Cloud, there are two CoreDNS pods, one is running on the control plane, and the other one is running on a worker node. Thus the information leak and `nf_conntrack` attacks both work on the Taikun cloud. First, a malicious container can disrupt other containers' non-local DNS queries in several seconds. Second, the victim container suffers 30% packet loss when connecting to the kube-apiserver. At the same time, the CoreDNS pod consumes the control plane 80% CPU resources. We show the victim's packet loss and control plane's CPU consumption in Figs. 4 and 5.

Attacking Node Exporter. Node Exporter is one of the Prometheus components designed to monitor the host system. By default, a node exporter pod exists on each node and exposes the metrics interface on the 9100 TCP port without any access restrictions or rate limitations. Thus, a malicious container can send non-restricted and rate-unlimited requests to the node exporter pod on the control plane, which has two consequences.

First, a malicious container can send a single request to the `node_exporter` pod running on the control plane, and get the control plane's sensitive information, e.g., the control plane's `nf_conntrack_count` (a resource counter of `nf_conntrack` table) and the `nf_conntrack_max` (the quota of `nf_conntrack` table). A malicious container can exploit the leaked information to fill up the control plane's `nf_conntrack` table.

Second, similar to the CoreDNS interface attack, by sending a large number of requests, a malicious container can make the control plane drop packets randomly and consume more CPU resources. At the same time, it makes the victim containers drop a lot of packets when communicating with the control plane.

In our local testbed, due to the node exporter attack, a victim container suffers 40% packet loss when connecting to the kube-apiserver, and the node exporter pod consumes control plane 80% CPU resources. In Alibaba cloud, as it secures the node exporter by HTTPS TLS, a malicious user cannot send requests to the node exporter service without authorization, and this attack fails in Alibaba Cloud.

In Taikun Cloud, the result of node exporter attack is even worse than in the local testbed. It is mainly because Taikun aggregates all nodes' node exporters under one Kubernetes service. More specifically, when a container sends a request to the `node-exporter-service-IP:9100/metrics`, the request will be forwarded to the node exporter pods running on each node randomly. Thus, for information leakage, by sending certain (malicious) requests, the node exporter service returns sensitive information about all nodes, not just the control plane. For the DoS attack, it fills up each node's `nf_conntrack` table, which makes the nodes in the cluster drop 40% packets and consumes 50% more CPU resources. We show the packet loss in Fig. 4 and CPU consumption in Fig. 5.

Attacking Grafana. Grafana is a popular dashboard UI that exposes metrics on 3000 TCP port without access restrictions or rate limitations. Similar to the former attacks, a malicious container can send

Table 4

The kube-apiserver's interfaces and corresponding verbs which a malicious container can use the default service account token to access.

Interfaces	Verbs
<code>selfsubjectreviews.authentication.k8s.io</code>	create
<code>selfsubjectaccessreviews.authorization.k8s.io</code>	create
<code>selfsubjectrulesreviews.authorization.k8s.io</code>	create
<code>/api/*</code>	get
<code>/api</code>	get
<code>/apis/*</code>	get
<code>/apis</code>	get
<code>/healthz</code>	get
<code>/livez</code>	get
<code>/openapi/*</code>	get
<code>/openapi</code>	get
<code>/readyz</code>	get
<code>/version/</code>	get
<code>/version</code>	get

multiple requests to `Grafana-SERVICE-IP:3000/metrics` and disrupt the control plane's functionalities. In our local environment, the victim container suffers 70% packet loss when sending requests to the kube-apiserver. The Grafana pod consumes the control plane 79% CPU resources. Neither Alibaba Cloud nor Taikun cloud uses the Grafana service. Therefore, this attack is not applicable to the both two cloud environments. We show these results in Figs. 4 and 5.

Attacking Calico. Calico is one of the most popular Kubernetes network add-ons. One of its components, `calico-kube-controller` exposes metrics on 9094 TCP port without access restrictions or rate limitations on the control plane. A malicious container can send a high volume of requests to the `calico-kube-controller`, which lead to two consequences.

First, it leaks the cluster's topology. Specifically, Kubernetes uses IPAM (IP Address Management) plugins to allocate and manage the IP addresses assigned to pods, and the metrics returned by `calico-kube-controller` include the Calico IPAM IP pool (Tigera, 2025) messages. For example, it returns `ipam_allocations_per_node`, which contains the node name and the number of IPs allocated to the pods running on each node. Thus, a malicious container can infer the cluster topology by node names, as well as the number of pods running on each node by the number of IPs.

Second, similar to the former attacks, a malicious container can send a large number of requests in several seconds, which fill up the control plane's `nf_conntrack` table and consume more CPU resources. In our local testbed, the victim container suffers 50% packet loss when connecting to the control plane. Besides, the `calico-kube-controller` consumes 80% of the control plane CPU resources. For Alibaba Cloud and Taikun cloud, they use Flannel instead of Calico. This attack is not applicable to these cloud environments. The results are shown in Figs. 4 and 5.

4.3. Case 2: Kube-apiserver attacks

The second case is the kube-apiserver attacks. In such an attack, a malicious container can exploit its service account token to generate a large number of requests to the kube-apiserver, and make it interact with other Kubernetes components (in Fig. 3). Thus, the kube-apiserver and other Kubernetes components can fill up the control plane's `nf_conntrack` table, which makes the control plane drop network packets randomly. In addition, the victim container experiences up to 50% packet loss when connecting to the control plane. Besides, all these components consume up to 90% more CPU resources.

Attack steps analysis. There are two steps in the kube-apiserver attacks. First, the malicious container sends a large number of requests to the kube-apiserver. Second, to handle the incoming requests, the kube-apiserver interacts with other Kubernetes components.

To achieve step one, we need to find all the service interfaces that kube-apiserver allows the container to access using the service account token. We use the default service account token as an example to illustrate the details. Kubernetes RBAC determines what interfaces a service account token can access. In the RBAC mechanism, the role or clusterrole contains rules that represent a set of permissions. A rolebinding or clusterrolebinding binds a role or a clusterrole to groups, users or serviceaccounts. More specifically, the clusterrolebindings owned by the default serviceaccount bind the `system:basic-user`, `system:discovery`, and `system:public-info-viewer` clusterroles, thus it is allowed to interact with fourteen service interfaces (as shown in Table 4). In step one, if one sends requests to the service interfaces and makes a large number of TCP connections, it will fill up the control plane's `nf_conntrack` table quickly. In this way, a malicious container can disrupt the control plane's network functionality.

To achieve step two, the requests sent to a specific interface should make the kube-apiserver interact with other components. We use the `/healthz` interface in Table 4 as an example to illustrate the details. The kube-apiserver provides `/healthz` to indicate the current status of the kube-apiserver. When receiving an HTTP GET request for `/healthz`, the kube-apiserver automatically performs a series of checks on multiple control plane components. For example, for the `etcd`, the kube-apiserver sends a request to get the value of key `/registry/health`, which is stored in the `etcd` database, and then it uses the response to determine if the `etcd` is healthy or not. Thus, a malicious container can send requests to the `/healthz` interface and make the kube-apiserver interact with other components. Besides the `/healthz` interface, the `/readyz` and `/livez` interfaces also have the similar consequences. Thus, the kube-apiserver interacts with other components through TCP connections. As a result, other components in the control plane can also contribute to filling the control plane's `nf_conntrack` table, similar to what the kube-apiserver does in step one. This can lead to the control plane dropping packets randomly and consuming more resources.

Experiments. We do the experiments in the local, Alibaba Cloud and Taikun Cloud environments. The results show that all of them are vulnerable to the kube-apiserver attacks. Specifically, in our local environment, by sending a large number of requests to the service interfaces shown in Table 4, five malicious containers can fill up the `nf_conntrack` table in five minutes, and these attacks make the victim container suffers 30% packet loss when connecting to the control plane. Besides, the kube-apiserver and other components consume 90% additional CPU resources of the control plane. In the Alibaba ACK environment, there are three control plane nodes, these attacks make each of them drops 20% packets and consume 70% additional CPU resources. In the Taikun Cloud environment, the victim container suffers 50% packet loss, and it consumes 80% additional CPU resources of the control plane. We show these results in Figs. 4 and 5.

4.4. Case 3: Pod readiness probe attack

The third case is to exploit the pod readiness probe, which involves attacking strategy ③. It consists of two steps, as shown in Fig. 6. First, malicious containers exploit the readiness probe to periodically change the status of the pods, causing the kubelet to synchronize these status changes to the kube-apiserver. After that, the kube-apiserver modifies all the workloads status related to the pods, and sends requests to the `etcd` to update all the data values. This step amplifies the effect of the attack, which makes the kube-apiserver and `etcd` consume up to 55.3% CPU resources.

Attack steps analysis. We use the readiness probe to achieve the first step. The kubelet leverages the readiness probe to determine when a container is ready to start accepting traffic (Kubernetes Authors, 2025b). More specifically, a readiness probe checks the container's

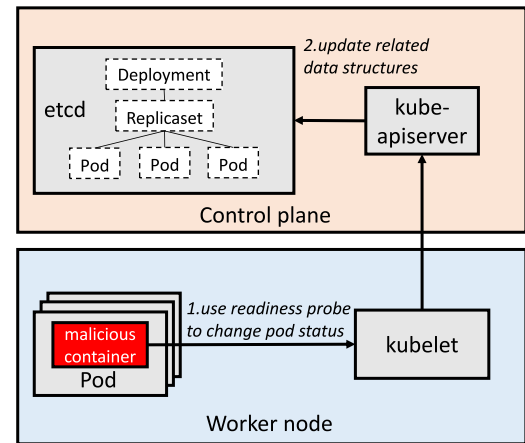


Fig. 6. The overview of the readiness probe attack.

status periodically, and changes the pod's status to ready or not according to the check result. When the pod's status changes, the kubelet synchronizes the pod's status to the kube-apiserver periodically. Therefore, we can exploit the readiness probe to make the kubelet patch the pod's status to the kube-apiserver.

To achieve the second step, the data dependencies of the pod status changes should make the kube-apiserver interact with other control plane components. We use the `pod↔replicaset↔deployment` data dependency as an example to illustrate the details (as shown in Fig. 6). Specifically, the `replicaset` is a Kubernetes workload which is used to manage a group of pods, and the `deployment` is an upper-level workload that manages the `replicasets`. When a single pod status change is received by the kube-apiserver, it modifies the pod and related `replicaset/deployment` status, and updates all these values to the `etcd` back end. Thus, the data dependency (i.e., `pod↔replicaset↔deployment`) amplifies the pod status changes, which makes the kube-apiserver and `etcd` consume more resources.

Experiments. In our local testbed, we run one hundred malicious containers, each of which is running in a pod managed by a separate `replicaset` and `deployment`. Each container leverages the readiness probe to modify its pods' ready status per second. As a result, the kube-apiserver and `etcd` consume 55.3% additional CPU resources of the control plane. In the Alibaba Cloud environment, there are three control plane nodes. This attack consumes each node 15.6% additional CPU resources. And in the Taikun Cloud environment, this attack consumes 51.2% additional CPU resources.

4.5. Case 4: Endpoints flooding attack

The fourth case exploits the endpoints status changes. In the attack, a malicious container leverages the kube-apiserver as a proxy, and sends endpoints changes to kube-proxy of all nodes in the cluster (④ in Fig. 3). As a result, it consumes up to 60% CPU resources of the control plane, and flushes all the node's iptables.

Attack steps analysis. This attack has two steps, as shown in Fig. 7. First, the malicious container sends a large number of requests to the kube-apiserver. Then, the incoming requests make the kube-apiserver components send requests to kube-proxy of all other nodes components. Here, we use the readiness probe described in Section 4.4 to achieve the first step.

To achieve the second step, the data dependencies of the request should make the kube-apiserver send messages to other worker nodes. We use the `pod↔endpoint↔service` data dependency as an example to illustrate the details. Note that a service is an abstract data structure which defines a logical set of pods and an IP address by which

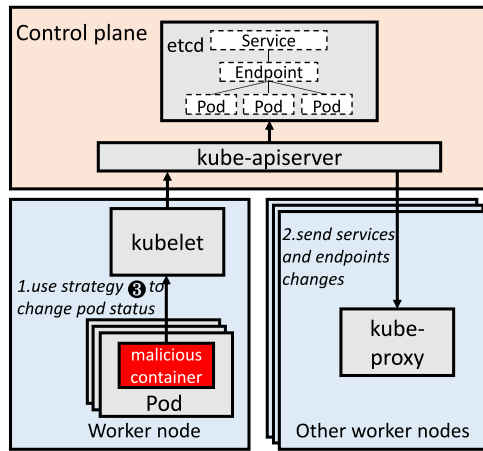


Fig. 7. The overview of the endpoints flooding attack.

to access them, while an endpoint is another structure that binds a set of pods to a specific service. As shown in Table 1, the kube-proxy watches the services/endpoints interfaces status change messages sent by the kube-apiserver. When the kube-proxy receives a large number of services/endpoints status changes sent by the kube-apiserver, it flushes the worker node's iptables rules accordingly and consumes worker node resources.

Thus, a malicious container can modify its pods status, and leverage the kube-apiserver as a proxy for sending related services/endpoints status changes to the kube-proxy on the worker nodes. After that, the kube-proxy flushes its local iptables. This data dependency (i.e., pod $\leftrightarrow$ endpoint $\leftrightarrow$ service) makes the control plane forward workloads to all other nodes in the cluster.

Experiments. Our experiment deploys one hundred malicious containers in one hundred pods on one worker node. Each container leverages the readinessProbe to modify its pod's ready status per second. The pods status changes invoke the kube-apiserver to send related service/endpoints status information to all other nodes' kube-proxy, which flushes all the nodes' iptables.

This attack works in all three environments. In the local environment, it consumes control plane 60% additional CPU resources, and the kube-proxy flushes all worker nodes' iptables, which is a flooding attack. In the Alibaba Cloud environment, this attack makes three control plane nodes consume up to 30% additional CPU and flush all nodes' iptables. In the Taikun Cloud environment, this attack makes the control plane consume up to 55% CPU and flush all nodes' iptables.

5. Discussion

In this section, first we discuss the practicality of our attacks. Then we give out some suggestions to mitigate the attacks.

5.1. Practicality discussion

We report our attacks to the relevant communities and cloud vendors. All the problems are confirmed by them. Further, we provide some suggestions to fix the problems.

Report to communities. For the third-party app attacks, we report the problems to the related communities. For example, we report information leaks and DoS attacks on CoreDNS to its community. We suggest either disabling the CoreDNS metrics service or using HTTPS to secure the metrics. They accept our suggestions and add the fix to their to-do list immediately. Other third-party app communities give us similar responses.

Specially, all the third-party apps are vulnerable to the metrics interfaces. This problem is somewhat brought by Prometheus collecting information from metrics. Thus, we report this problem to the Prometheus community. They confirm this problem. Further, we suggest using HTTPS TLS to secure all these metrics endpoints. They accept our suggestion and say: "What is more, the problem here is that a number of users are not aware of the implications of exposing metrics via an HTTP endpoint". They plan to fix this problem by improving Prometheus' documentation, which clarifies the threat model and uses our recommendation to secure the metrics endpoints.

For other attacks related to Kubernetes components, we report these issues to the Kubernetes community. The community confirms all the problems. Further, we suggest fixing the problems by adding more rate limitations per namespace. The community accept our suggestion and will fix the source code later.

Report to cloud vendors. Besides the related communities, we report these issues to cloud vendors. Alibaba Cloud and Taikun Cloud both confirm these problems. Furthermore, Taikun accepts our suggestions and will add more restrictions to their productions. Alibaba Cloud confirms these issues and offers some suggestions to mitigate the risks. However, they admit that none of their recommendations can solve these problems fundamentally. Besides, Alibaba Cloud's security reinforcements are based on community architectures and theories. Thus, with relevant updates from the communities, these issues will eventually be addressed on the Alibaba Cloud. Taikun Cloud confirms these problems, and they are trying to fix them with some additional restrictions on accessing the control plane's endpoints. Further, they say: "So far it is still in dev but will eventually be ported to prod".

Report summary. All these communities and cloud vendors confirm these problems. The control plane interface attacks we figured out need both users and cloud vendors to review their Kubernetes configurations and add more restrictions. We hope this paper can raise broader awareness of these threats among users and cloud vendors.

5.2. Mitigation discussion

In the following, we give out multiple strategies to mitigate the risks introduced by the control plane interface attacks.

Disabling unnecessary interfaces. Disabling those unnecessary interfaces can effectively reduce the attack surface. Specifically, the Kubernetes administrator should disable those metrics interfaces to avoid information leaks and DoS attacks. However, the side effect is that it will reduce the monitoring metrics. It is a trade-off between the cluster monitorability and the security.

Securing all interfaces. Using HTTPS TLS to harden the interface is a way to reduce unauthorized accesses. For those security-critical interfaces, users and cloud vendors should secure these interfaces and only allow necessary accesses. However, it may introduce additional performance overhead.

Applying more restricted network policies. Users and cloud vendors should specify network policies to control network traffic at the IP address or port level. For sensitive interfaces such as metrics, network policies should be specified and it only allows necessary pods to communicate with the Kubernetes components and third-party apps. However, it may bring unexpected additional efforts to provide the exact network policy in a large-scale Kubernetes cluster.

Improving the current monitoring tools. Developers of current monitoring tools should review their design. More specifically, all third-party apps expose the metrics with HTTP plain text, which is used by Prometheus by default. We believe that monitoring tools that use similar mechanisms are vulnerable to our attacking strategy.

6. Related work

In the field of Kubernetes security, numerous researchers have contributed to the understanding of various threats (Curtis and Eisty, 2024). Shringarputale et al. (2020) confirm attackers can launch co-residency attacks to a victim application from inside state-of-the-art containers running in the same Kubernetes cluster. Shamim (2021) studies attacks under the scenario of violating Kubernetes security best practices. Ben David and Bremner Barr (2021) propose a DDoS attack against Kubernetes auto-scaling. Chamberlain et al. (2025) investigate the Economic Denial of Sustainability (EDoS) attacks on Kubernetes. Pecka et al. (2022) study the attack methods in the DevOps pipeline with a Kubernetes environment. Yang et al. (2021a) propose abstract resource attacks in Kubernetes container environments. Xiao et al. (2023) propose operation forwarding attacks which can break the isolation between containers within Kubernetes. Knob et al. (2024) exploit a series of design flaws in the Kubernetes CRI-API to execute a DoS attack. Xu et al. (2024) make an empirical study on Kubernetes Operator bugs. Chen et al. (2025) present the Cross-Namespace Reference Vulnerability, which prevents Kubernetes from properly isolating namespaces. Yang et al. (2023) discuss how to exploit the excessive permissions granted to third-party apps in Kubernetes to achieve privilege escalation within Kubernetes clusters.

There are also works on securing Kubernetes (Spahn et al., 2023). Kong et al. (2019) propose a secure container deployment strategy to defend against co-resident attacks in container cloud. Karn et al. (2020) realize the detection of malicious cryptomining software running in Kubernetes cluster by monitoring Linux system calls. Baarzi et al. (2020) implement a system for defending application-layer DoS attacks against microservices. Haque et al. (2022) propose a scheme to automate the security configuration of container orchestrator. Blaise and Rebecchi (2022) propose a methodology for evaluating the security and extracting the risk mode of a Helm Chart deployment. Kermabon-Bobinnet et al. (2024) propose a performance profiling-based proactive security policy enforcement solution in Kubernetes environments. Malul et al. (2024) propose a comprehensive and adaptive, LLM-based method which can detect a wide variety of misconfigurations in Kubernetes. Shen et al. (2024) design and implement a flexible abstract resource confinement framework to achieve flexible and efficient abstract resource confinement in Kubernetes. Gu et al. (2025) propose a novel approach, namely EPScan, which automatically detects exploitable excessive permissions in third-party applications within Kubernetes. Tang et al. (2025, 2024a) and Tang et al. (2024) provide valuable insights into low-rate DoS (LDoS) attack mitigation strategies in software-defined networks (SDNs), which can inspire the design of defenses in container orchestration environments.

To the best of our knowledge, our work is the first to systematically exploit the control plane interfaces of Kubernetes. Our work further complements previous research efforts on understanding the security of container orchestration systems.

7. Conclusions and future work

This paper presented a systematic investigation of the security weaknesses in Kubernetes control plane interfaces. Our study revealed that several interfaces lack adequate access restrictions or rate limitations. To explore these weaknesses, we proposed four strategies based on different access paths and validated them through seven representative attacks across four case studies. The evaluation demonstrated that the control plane leaks information about cluster resource limits and topology, occasionally drops packets, and drives the CPU utilization of the control plane node up to 90%. Furthermore, other containers are unable to perform non-local DNS queries and may suffer up to 70% packet loss. These vulnerabilities were observed in both local and cloud environments, and we have responsibly disclosed them to the relevant communities and vendors. We also provided

practical recommendations for Kubernetes users, cloud providers, and developers.

Our work lays the foundation for further work on the systematic exploration of control plane vulnerabilities. One direction is to extend beyond the representative strategies studied here by developing automated analysis frameworks, such as static analysis combined with symbolic execution, to achieve broader coverage of insecure interfaces. Another direction is to design safe and detached experimental environments that allow indirect evaluation of these attacks in multi-tenant PaaS/container service settings. Such efforts will enable a deeper understanding of Kubernetes security and support the development of stronger defense mechanisms.

CRediT authorship contribution statement

Chen Wang: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Hongbo Tang:** Validation, Supervision, Resources. **Yu Zhao:** Validation, Investigation, Data curation. **Wei You:** Validation, Project administration, Investigation, Data curation. **Jie Yang:** Validation, Project administration, Investigation, Data curation. **Hang Qiu:** Validation, Supervision, Project administration, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the National Key Research and Development Program of China (Grant No. 2022YFB2902204), the Key Research and Development Program of Henan Province (Grant No. 231111211000) and Top Talent Development Program of Henan Province (Grant No. 244500510012).

Data availability

The authors do not have permission to share data.

References

- Addula, S.R., Ali, A., 2025. A novel permissioned blockchain approach for scalable and privacy-preserving iot authentication. *J. Cyber Secur. Risk Audit.* 2025, 222–237.
- Aljumaiah, O., Jiang, W., Addula, S.R., Almaiah, M.A., 2025. Analyzing cybersecurity risks and threats in it infrastructure based on nist framework. *J. Cyber Secur. Risk Audit.* 2025, 12–26.
- Almanasir, R., Al-solomon, D., Indrawes, S., Almaiah, M., Islam, U., Alshar'e, M., 2025. Classification of threats and countermeasures of cloud computing. *J. Cyber Secur. Risk Audit.* 2025, 27–42.
- Baarzi, A.F., Kesidis, G., Fleck, D., Stavrou, A., 2020. Microservices made attack-resilient using unsupervised service fissioning. In: *Proceedings of the 13th European Workshop on Systems Security*. pp. 31–36.
- Ben David, R., Bremner Barr, A., 2021. Kubernetes autoscaling: Yoyo attack vulnerability and mitigation. *arXiv e-prints*, arXiv:2105.
- Blaise, A., Rebecchi, F., 2022. Stay at the helm: Secure kubernetes deployments via graph generation and attack reconstruction. In: *2022 IEEE 15th International Conference on Cloud Computing. CLOUD, IEEE*, pp. 59–69.
- Chamberlain, J., Zheng, J., Zhu, Z., Liu, Z., Starobinski, D., 2025. Exploiting kubernetes autoscaling for economic denial of sustainability. *Proc. ACM Meas. Anal. Comput. Syst.* 9, 1–29.
- Chen, A., Jin, Z., Guo, Z., Chen, Y., 2025. Breaking the bulkhead: Demystifying cross-namespace reference vulnerabilities in kubernetes operators. *arXiv preprint arXiv:2507.03387*.
- CNCF, 2025a. Cloud native 2024: Approaching a decade of code, cloud, and change. <https://www.cncf.io/reports/cncf-annual-survey-2024/>. (Accessed 21 September 2025).
- CNCF, 2025b. Cloud native landscape guide: Certified kubernetes - hosted. <https://landscape.cncf.io/guide#platform--certified-kubernetes-hosted>. (Accessed 21 September 2025).

- CNCF, 2025c. Platform. <https://landscape.cncf.io/guide#platform>. (Accessed 21 September 2025).
- Curtis, J.A., Eisty, N.U., 2024. The kubernetes security landscape: Ai-driven insights from developer discussions. arXiv preprint [arXiv:2409.04647](https://arxiv.org/abs/2409.04647).
- David, R.B., 2021. Kubernetes Auto-Scaling: YoYo Attack Vulnerability and Mitigation (Master's thesis). Reichman University (Israel).
- Gu, Y., Tan, X., Zhang, Y., Gao, S., Yang, M., 2025. Epscan: Automated detection of excessive rbac permissions in kubernetes applications. In: 2025 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 3199–3217.
- Haq, M.U., Kholoosi, M.M., Babar, M.A., 2022. Kgsecconfig: A knowledge graph based approach for secured container orchestrator configuration. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 420–431.
- Islam Shamim, M.S., Ahamed Bhuiyan, F., Rahman, A., 2020. Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices. In: 2020 IEEE Secure Development. SecDev, pp. 58–64. <http://dx.doi.org/10.1109/SecDev45635.2020.00025>.
- Karn, R.R., Kudva, P., Huang, H., Suneja, S., Elfadel, I.M., 2020. Cryptomining detection in container clouds using system calls and explainable machine learning. IEEE Trans. Parallel Distrib. Syst. 32, 674–691.
- Kermabon-Bobinnee, H., Bagheri, S., GholipourChoubbeh, M., Majumdar, S., Jarraya, Y., Wang, L., Pourzandi, M., 2024. PerfSpec: Performance profiling-based proactive security policy enforcement for containers. IEEE Trans. Dependable Secur. Comput. 1–18. <http://dx.doi.org/10.1109/TDSC.2024.3420712>.
- Knob, L.A.D., Franzil, M., Siracusa, D., 2024. Exploiting kubernetes' image pull implementation to deny node availability. arXiv preprint [arXiv:2401.10582](https://arxiv.org/abs/2401.10582).
- Kong, T., Wang, L., Ma, D., Xu, Z., Yang, Q., Chen, K., 2019. A secure container deployment strategy by genetic algorithm to defend against co-resident attacks in cloud computing. In: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems. HPCC/SmartCity/DSS, IEEE, pp. 1825–1832.
- Kubernetes Authors, 2025a. Communication between nodes and the control plane — kubernetes. <https://kubernetes.io/docs/concepts/architecture/control-plane-node-communication/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025b. Define readiness probes — kubernetes. <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#define-readiness-probes> (Accessed 21 September 2025).
- Kubernetes Authors, 2025c. Installing addons. <https://kubernetes.io/docs/concepts/cluster-administration/addons/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025d. Kubeadm — kubernetes. <https://kubernetes.io/docs/reference/setup-tools/kubeadm/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025e. Pod security standards — kubernetes. <https://kubernetes.io/docs/concepts/security/pod-security-standards/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025f. Restrict a container's syscalls with seccomp — kubernetes. <https://kubernetes.io/docs/tutorials/security/seccomp/#enable-the-use-of-runtime-default-as-the-default-seccomp-profile-for-all-workloads> (Accessed 21 September 2025).
- Kubernetes Authors, 2025g. Securing a cluster. <https://kubernetes.io/docs/tasks/administer-cluster/securing-a-cluster/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025h. Service accounts — kubernetes. <https://kubernetes.io/docs/concepts/security/service-accounts/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025i. Using rbac authorization — kubernetes. <https://kubernetes.io/docs/reference/access-authn-authz/rbac/> (Accessed 21 September 2025).
- Kubernetes Authors, 2025j. Workloads — kubernetes. <https://kubernetes.io/docs/concepts/workloads/> (Accessed 21 September 2025).
- Malul, E., Meidan, Y., Mimran, A., Elovici, Y., Shabtai, A., 2024. Genkubsec: Llm-based kubernetes misconfiguration detection, localization, reasoning, and remediation. arXiv preprint [arXiv:2405.19954](https://arxiv.org/abs/2405.19954).
- Mohd Satar, S., Hussin, M., Mohamed, M.A., Abd Hamid, N., Kadir, M., Muda, R., Samual, J., 2025. Secure access control using ciphertext policy attribute-based encryption with performance optimization in cloud computing. J. Cyber Secur. Risk Audit. 2025, 287–305. <http://dx.doi.org/10.63180/jcsra.thestap.2025.4.8>.
- Pecka, N., Ben Othmane, L., Valani, A., 2022. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment. In: Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering. pp. 45–49.
- Shamim, S.I., 2021. Mitigating security attacks in kubernetes manifests for security best practices violation. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1689–1690.
- Shen, W., Wu, Y., Yang, Y., Liu, Q., Yang, N., Li, J., Lu, K., Ma, J., 2024. Towards understanding and defeating abstract resource attacks for container platforms. IEEE Trans. Dependable Secur. Comput..
- Shringarputale, S., McDaniel, P., Butler, K., La Porta, T., 2020. Co-residency attacks on containers are real. In: Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop. pp. 53–66.
- Spahn, N., Hanke, N., Holz, T., Kruegel, C., Vigna, G., 2023. Container orchestration honeypot: Observing attacks in the wild. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. pp. 381–396.
- Tang, D., Dai, R., Yan, Y., Li, K., Liang, W., Qin, Z., 2024a. When sdn meets low-rate threats: A survey of attacks and countermeasures in programmable networks. ACM Comput. Surv. 57, 1–32.
- Tang, D., Dai, R., Zuo, C., Chen, J., Li, K., Qin, Z., 2025. A low-rate dos attack mitigation scheme based on port and traffic state in sdn. IEEE Trans. Comput..
- Tang, D., Liu, B., Li, K., Xiao, S., Liang, W., Zhang, J., 2024. Pluto: A robust Idos attack defense system executing at line speed. IEEE Trans. Dependable Secur. Comput..
- Tigera, 2025. Get started with ip address management — calico documentation. <https://projectcalico.docs.tigera.io/networking/get-started-ip-addresses>. (Accessed 21 September 2025).
- Xiao, J., Yang, N., Shen, W., Li, J., Guo, X., Dong, Z., Xie, F., Ma, J., 2023. Attacks are forwarded: Breaking the isolation of {MicroVM-based} containers through operation forwarding. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 7517–7534.
- Xu, Q., Gao, Y., Wei, J., 2024. An empirical study on kubernetes operator bugs. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1746–1758.
- Yang, N., Shen, W., Li, J., Liu, X., Guo, X., Ma, J., 2023. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 3048–3062.
- Yang, N., Shen, W., Li, J., Yang, Y., Lu, K., Xiao, J., Zhou, T., Qin, C., Yu, W., Ma, J., et al., 2021a. Demons in the shared kernel: Abstract resource attacks against os-level virtualization. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 764–778.
- Yang, Y., Shen, W., Ruan, B., Liu, W., Ren, K., 2021b. Security challenges in the container cloud. In: 2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications. TPS-ISA, IEEE, pp. 137–145.

Chen Wang received his M.S. degree in Information and Communication Engineering from Information Engineering University in 2018. He is currently working toward the Ph.D. degree in Information Engineering University, Zhengzhou, China. His research interests include container security, next-generation mobile communication, and cyberspace security.

Hongbo Tang received the B.E. degree from Huazhong University of Science and Technology in 1989, and M.E. degree in Engineering from National University of Defense Technology in 1995. He is now a professor of Information Engineering University, China since 2006. His current interests include next-generation mobile communication and security.

Yu Zhao received his Ph.D. degree from Information Engineering University, Zhengzhou, China. His research interests include next-generation mobile communication, 5G Core Network Security.

Wei You received his Ph.D. degree from Information Engineering University, Zhengzhou, China. He is currently an associate professor, His research interests include next-generation mobile communication, Multi-Access Edge Computing, 5G network security.

Jie Yang received his Ph.D. degree from Information Engineering University in 2022. Currently, he is an assistant research fellow of Information Engineering University. His major research interests include wireless communication security and network functions virtualization.

Hang Qiu received M.S. and Ph.D. degrees in Information and Communication Engineering from the Information Engineering University in 2019 and 2022, respectively. He is now an assistant researcher of the Information Engineering University, China since 2022. His research interests include mobile communication network architecture, network function virtualization, and network cyberspace security.